

Introduction to Kernel Modules on Zynq Linux System

Table Of Contents

Introduction	2
Procedure	2
1. SD Card Mounting and Verification	2
2. PetaLinux Setup and Project Creation	2
3. Creation of Hello Kernel Module	3
4. Creation of Multiply Kernel Module	3
5. Implementation of Hardware Access	3
6. Build and Execution	3
Results	4
Conclusion	4
Knowledge Transfer	5
Appendix	6
APPENDIX A (SD Card Mounting and Unmounting)	6
APPENDIX B (PetaLinux Build Success).....	7
APPENDIX C (Module Outputs)	8
APPENDIX D (Kernel Logs and Module Operations).....	8
APPENDIX E (hello.c)	9
APPENDIX F(multiply.c)	10

Introduction

The objective of this laboratory exercise was to **construct and load Linux kernel modules on the Zybo Z7-10 board** running a PetaLinux-based system. This lab focused on developing a basic “Hello World!” kernel module to understand module creation, compilation, and insertion into the Linux kernel, as well as implementing a second kernel module to interact with the multiplication peripheral designed in a previous lab. Unlike the previous lab where an existing PetaLinux project was reused, in this lab the PetaLinux environment and project were reinitialized to ensure a clean setup for kernel module development. This exercise provided practical exposure to **kernel-level programming** and demonstrated how software running in kernel space can access hardware peripherals using memory-mapped I/O, with the multiply module reading from and writing to hardware registers and printing the results to the kernel message buffer.

Procedure

1. SD Card Mounting and Verification

1. The Zybo Z7-10 board was booted using the SD card containing the required boot files (BOOT.bin, image.ub, and boot.scr) prepared in the previous lab.
2. The SD card was mounted using:
`mount /dev/mmcblk0p1 /mnt/`
3. The contents were verified using:
`cd /mnt`
`ls -la`
4. A directory was created in /mnt to verify write capability.
5. The SD card was unmounted using:
`cd /`
`umount /mnt`

The mounting and unmounting process is shown in [Appendix A](#).

2. PetaLinux Setup and Project Creation

6. A PetaLinux workspace was created under /tmp/petalinux_nithingowtham25, and the PetaLinux SDK was installed following the steps from the previous lab.
7. The PetaLinux environment was initialized by sourcing:
`source petalinux_install/settings.sh`
8. A new PetaLinux project was created using the Zynq template:
`petalinux-create -t project --template zynq -n linux_boot_nithingowtham25 --tmpdir /tmp/petalinux_nithingowtham25/tmp_dir`
9. The hardware description file (.xsa) generated in the previous lab was used to configure the project:
`petalinux-config --get-hw-description=/home/grads/n/nithingowtham25/Spring2026/ECEN749/lab4_new/linux_boot_wrapper.xsa --silentconfig`

3. Creation of Hello Kernel Module

10. A module named hello was created:

```
petalinux-create -t modules --name hello --enable
```

11. The provided hello.c code was added to the module directory.

12. The module was compiled and the generated hello.ko file was copied to the SD card.

The source code for the hello module is shown in [Appendix E](#).

The successful build process is shown in [Appendix B](#).

4. Creation of Multiply Kernel Module

13. A module named multiply was created:

```
petalinux-create -t modules --name multiply --enable
```

14. Header files xparameters.h and xparameters_ps.h were copied from

“lab3/multiply_wrapper/zynq_fsbl/zynq_fsbl_bsp/ps7_cortexa9_0/include” into the module directory “multiply/files/”.

15. The multiply.bb file was modified to include the header files.

16. The skeleton code was copied into multiply.c, and necessary function calls were added to implement hardware interaction.

The source code for the multiply module is shown in [Appendix F](#).

5. Implementation of Hardware Access

17. The physical address of the multiplier was defined using macros from xparameters.h.

18. The ioremap() function was used to map the physical address to a virtual address.

19. The iowrite32() function was used to write values to hardware registers.

20. The ioread32() function was used to read values from the registers.

21. A printk statement was added to display both physical and virtual addresses.

22. The iounmap() function was used during module removal to release allocated resources.

6. Build and Execution

23. Both modules were compiled using:

```
petalinux-build
```

24. The .ko files were copied to the SD card and reinserted into the Zybo board.

25. The SD card was mounted again:

```
mount /dev/mmcblk0p1 /mnt/
```

26. The modules were loaded into the kernel:

```
insmod /mnt/hello.ko  
insmod /mnt/multiply.ko
```

27. The output was viewed using: `dmesg | tail`

28. The modules were verified using `lsmod` and removed using `rmmod`.

The module outputs are shown in [Appendix C](#).

Kernel logs and module operations (`dmesg`, `lsmod`, `rmmod`) are shown in [Appendix D](#).

(The complete working of the system was demonstrated to the TA)

Results

Linux was successfully booted on the Zybo Z7-10 board, and kernel modules were compiled and loaded correctly. The hello module executed as expected and printed “**Hello world!**” upon insertion into the kernel, while “**Goodbye world!**” was displayed when the module was removed, as shown in [Appendix C](#). The multiply module successfully interfaced with the hardware multiplication peripheral. The module mapped the physical address of the multiplier to a virtual address using `ioremap()`, enabling safe access within the Linux virtual memory space. The values 7 and 2 were written into the input registers using `iowrite32()`, and the output register returned the correct multiplication result of **14**, as observed in [Appendix C](#).

The following key changes were implemented in the multiply module:

- Use of `ioremap()` for address translation
- Use of `iowrite32()` and `ioread32()` for register access
- Use of `printk()` for displaying results

All outputs were observed using the picocom serial console and verified using the `dmesg` command. Additional kernel logs, module listing (`lsmod`), and module removal operations (`rmmod`) are shown in [Appendix D](#).

The source code for both `hello.c` and `multiply.c` is included in [Appendix E](#) and [Appendix F](#) respectively. The successful execution of both modules was demonstrated to the TA.

Conclusion

This laboratory exercise provided valuable experience in Linux kernel module development and hardware interaction in an embedded system. The hello module introduced the structure and lifecycle of kernel modules, while the multiply module demonstrated how kernel modules can be used to interact with hardware peripherals. The use of `ioremap()` emphasized the importance of **virtual memory and address translation** in Linux systems. Additionally, the ability to read from and write to hardware registers using **kernel-level APIs** highlighted the interaction between software and hardware in embedded systems. Overall, this lab strengthened understanding of system-level programming and kernel customization, providing a strong foundation for developing advanced device drivers and embedded Linux applications.

Knowledge Transfer

- a) **What is the mount point for the SD card on the Ubuntu machine? Hint: Where does the SD card lie in the directory structure of the Ubuntu file system? This should be different from the Linux system on the Zybo board.**

On the **Ubuntu machine**, the SD card is automatically mounted under the /media directory, typically in the path **/media/<username>/<device-name>**. This mounting location is managed by the Ubuntu operating system through its **automatic mounting** mechanism and is not explicitly specified or enforced by any commands in the lab manual. In my case, I renamed the device as “lab5_nithin” and the SD card was mounted at **/media/nithingowtham25/lab5_nithin**.

In contrast, on the Linux system running on the **Zybo Z7-10 board**, the SD card is **manually mounted** at **/mnt** using the `mount /dev/mmcblk0p1 /mnt/` command as instructed in the lab manual. This highlights the difference between automatic mounting in a desktop Linux environment and manual mounting in an embedded Linux system.

- b) **Why do we need to use ioremap to map the virtual address before reading from or writing to the multiplier?**

The use of `ioremap()` is necessary because the Linux kernel operates in a virtual memory space and cannot directly access physical memory addresses defined by hardware. The multiplication peripheral resides at a fixed physical address, but before any read or write operations can be performed, this address must be mapped to a corresponding virtual address. The `ioremap()` function performs this translation, enabling the kernel to safely access memory-mapped I/O regions. Without this mapping, any attempt to access the hardware registers would result in invalid memory access or potential system instability. The `iounmap()` function is used to release this mapping during module cleanup.

Appendix

APPENDIX A (SD Card Mounting and Unmounting)

- Mounting SD card on Zybo board
- Verifying contents using ls
- Unmounting using umount

```
linux_boot_nithingowtham25:/mnt# mkdir test
linux_boot_nithingowtham25:/mnt# ls -la
drwxrwx---   3 root    disk      8192 Jan  1  1970 .
drwxr-xr-x  17 root    root        0 Mar  9 12:34 ..
-rwxrwx---   1 root    disk     3212928 Mar  5  2026 BOOT.BIN
-rwxrwx---   1 root    disk      3016 Mar  5  2026 boot.scr
-rwxrwx---   1 root    disk    17841212 Mar  5  2026 image.ub
drwxrwx---   2 root    disk      8192 Mar  9 12:37 test
linux_boot_nithingowtham25:/mnt# cd /
linux_boot_nithingowtham25:/# umount /m
media/ mnt/
linux_boot_nithingowtham25:/# umount /mnt/
linux_boot_nithingowtham25:/# dc /mnt/
linux_boot_nithingowtham25:/# cd /m
media/ mnt/
linux_boot_nithingowtham25:/# cd /mnt/
linux_boot_nithingowtham25:/mnt# ls
linux_boot_nithingowtham25:/mnt# ls -la
drwxr-xr-x   2 root    root        0 Mar  9 12:34 .
drwxr-xr-x  17 root    root        0 Mar  9 12:34 ..
linux_boot_nithingowtham25:/mnt#
```

APPENDIX B (PetaLinux Build Success)

- Successful PetaLinux build output
- Compilation of kernel modules

```
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25...
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25...
0: glibc-locale-2.36-r0 do_package - 1m2s (pid 1200954) 99% |#####|
Setscene tasks: 1571 of 1571
Currently 21 running tasks (2878 of 4184) 68% |#####|
0: glibc-locale-2.36-r0 do_package - 1m2s (pid 1200954) 99% |#####|
Setscene tasks: 1571 of 1571
Currently 21 running tasks (2879 of 4184) 68% |#####|
0: glibc-locale-2.36-r0 do_package - 1m2s (pid 1200954) 99% |#####|
Setscene tasks: 1571 of 1571
Currently 20 running tasks (2879 of 4184) 68% |#####|
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
WARNING: hello-1.0-r0 do_package_qa: QA Issue: Recipe LICENSE includes obsolete license
s GPLv2 [obsolete-license]
WARNING: multiply-1.0-r0 do_package_qa: QA Issue: Recipe LICENSE includes obsolete lice
nses GPLv2 [obsolete-license]
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
Setscene tasks: 1571 of 1571
NOTE: Tasks Summary: Attempted 4184 tasks of which 437 didn't need to be rerun and all
succeeded.

Summary: There were 3 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nithinithingo
wtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nithinithingowtham25@
zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-ninithingowtham25@zach-333-em
4-07:/tmp/petalinux_nithingowtham25/linux_boot-nithnithingowtham25@zach-333-em4-07:/tmp
/petalinux_nithingowtham25/linux_bootnithingowtham25@zach-333-em4-07:/tmp/petalinux_nit
hingowtham25/linux_bonithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/lin
ux_nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nnithingow
tham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nnithingowtham25@zach-
333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nnithingowtham25nithingownithingow
nithingowtham25@zach-333-em4-07:/tmp/petalinux_nithingowtham25/linux_boot-nithintham25$
gowtham25$
```

APPENDIX C (Module Outputs)

- Hello module output (Hello world!)
- Multiply module output (register reads and result = 14)

```
linux_boot_nithingowtham25:/# insmod /mnt/hello.ko
hello: loading out-of-tree module taints kernel.
Hello world!
```

```
linux_boot_nithingowtham25:/# insmod /mnt/multiply.ko
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = 24fe5623
Writing a 7 to register 0
Writing a 2 to register 1
Read 7 from register 0
Read 2 from register 1
Read 14 from register 2
```

APPENDIX D (Kernel Logs and Module Operations)

- dmesg | tail output

```
linux_boot_nithingowtham25:/# dmesg | tail
hello: loading out-of-tree module taints kernel.
Hello world!
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = 24fe5623
Writing a 7 to register 0
Writing a 2 to register 1
Read 7 from register 0
Read 2 from register 1
Read 14 from register 2
```

- lsmod output

```
linux_boot_nithingowtham25:/# lsmod
Tainted: G
multiply 16384 0 - Live 0xbf00a000 (0)
hello 16384 0 - Live 0xbf005000 (0)
uio_pdrv_genirq 16384 0 - Live 0xbf000000
```

- rmmod hello followed by lsmod

```
linux_boot_nithingowtham25:/# rmmod hello
Goodbye world!
linux_boot_nithingowtham25:/# lsmod
Tainted: G
multiply 16384 0 - Live 0xbf00a000 (0)
uio_pdrv_genirq 16384 0 - Live 0xbf000000
linux_boot_nithingowtham25:/#
```


- `rmmod multiply`

```
linux_boot_nithingowtham25:/# rmmod multiply
unmapping virtual address space....
linux_boot_nithingowtham25:/#
```

- Final `dmesg` output

```
linux_boot_nithingowtham25:/# dmesg | tail
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = 24fe5623
Writing a 7 to register 0
Writing a 2 to register 1
Read 7 from register 0
Read 2 from register 1
Read 14 from register 2
Goodbye world!
unmapping virtual address space....
linux_boot_nithingowtham25:/#
```

APPENDIX E (hello.c)

```
Z: > Spring_2026 > ECEN749 > lab5 > petalinux_nithingowtham25 > linux_boot_nithingowtham25 > project-spec > meta-user > recipes-modules > hello > files > C hello.c > ...
1  /*
2   * hello.c - Hello World kernel module
3   *
4   * Demonstrates module initialization, module release, and printk.
5   *
6   * (Adapted from various example modules including those found in the
7   * Linux Kernel Programming Guide, Linux Device Drivers book, and
8   * FSM's device driver tutorial)
9   */
10
11 #include <linux/module.h> /* Needed by all modules */
12 #include <linux/kernel.h> /* Needed for KERN_INFO */
13 #include <linux/init.h> /* Needed for __init and __exit macros */
14
15 /*
16  * This function is run upon module load.
17  * This is where you setup data structures and reserve resources.
18  */
19 static int __init my_init(void)
20 {
21     printk(KERN_INFO "Hello world!\n");
22     return 0; /* Non-zero return means init failed */
23 }
24
25 /*
26  * This function is run just prior to module removal.
27  * Release all resources here.
28  */
29 static void __exit my_exit(void)
30 {
31     printk(KERN_ALERT "Goodbye world!\n");
32 }
33
34 /* Module metadata */
35 MODULE_LICENSE("GPL");
36 MODULE_AUTHOR("ECEN749 Student: Nithin Gowtham");
37 MODULE_DESCRIPTION("Simple Hello World Module");
38
39 /* Register init and exit functions */
40 module_init(my_init);
41 module_exit(my_exit);
```

APPENDIX F(multiply.c)

```

Z: > Spring_2026 > ECEN749 > lab5 > petalinux_nithingowtham25 > linux_boot_nithingowtham25 > project-spec > meta-user > recipes-modules > multiply > files > C multiply.c > ...
1  #include <linux/module.h> /* Needed by all modules */
2  #include <linux/kernel.h> /* Needed for KERN_* and printk */
3  #include <linux/init.h> /* Needed for init and exit macros */
4  #include <asm/io.h> /* Needed for IO reads and writes */
5  #include "xparameters.h" /* needed for physical address of multiplier */
6
7  /* from xparameters.h */
8  #define PHY_ADDR XPAR_MULTIPLY_0_S00_AXI_BASEADDR /* physical address of multiplier */
9
10 /* size of physical address range for multiply */
11 #define MEMSIZE (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR - XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)
12
13 void *virt_addr; /* virtual address pointing to multiplier */
14
15 /* This function is run upon module load. This is where you setup data
16 structures and reserve resources used by the module. */
17 static int __init my_init(void)
18 {
19     /* Linux kernel's version of printf */
20     printk(KERN_INFO "Mapping virtual address...\n");
21
22     /* map virtual address to multiplier physical address */
23     /* use ioremap */
24     virt_addr = ioremap(PHY_ADDR, MEMSIZE);
25
26     if (!virt_addr) {
27         printk(KERN_ALERT "ioremap failed!\n");
28         return -ENOMEM;
29     }
30
31     /* print physical and virtual addresses */
32     printk(KERN_INFO "Physical address = %x\n", PHY_ADDR);
33     printk(KERN_INFO "Virtual address = %p\n", virt_addr);
34
35     /* write 7 to register 0 */
36     printk(KERN_INFO "Writing a 7 to register 0\n");
37     iowrite32(7, virt_addr + 0); /* base address + offset */
38
39     /* Write 2 to register 1 */
40     printk(KERN_INFO "Writing a 2 to register 1\n");
41     iowrite32(2, virt_addr + 4); /* use iowrite32 */
42
43     printk(KERN_INFO "Read %d from register 0\n", ioread32(virt_addr + 0));
44     printk(KERN_INFO "Read %d from register 1\n", ioread32(virt_addr + 4));
45     printk(KERN_INFO "Read %d from register 2\n", ioread32(virt_addr + 8));
46
47     /* A non 0 return means init module failed; module can't be loaded */
48     return 0;
49 }
50
51 /* This function is run just prior to the module's removal from the
52 system.
53 You should release ALL resources used by your module
54 here (otherwise be prepared for a reboot). */
55 static void __exit my_exit(void)
56 {
57     printk(KERN_ALERT "unmapping virtual address space...\n");
58
59     iounmap((void *)virt_addr);
60 }
61
62 /* These define info that can be displayed by modinfo */
63 MODULE_LICENSE("GPL");
64 MODULE_AUTHOR("ECEN749 Student: Nithin Gowtham");
65 MODULE_DESCRIPTION("Simple multiplier module");
66
67 /* Here we define which functions we want to use for initialization
68 and cleanup */
69 module_init(my_init);
70 module_exit(my_exit);
71

```